Indira Gandhi
National Open University
School of Computer and
Information Sciences

BCSL - 159
INTRODUCTION TO ALGORITHM DESIGN LAB

LAB MANUAL

SECTION 1
INTRODUCTION TO ALGORITHM DESIGN

BLOCK INTRODUCTION

This lab course has been designed for providing youwith hands on experience on Algorithm Design using C Programming. You have studied the theoretical aspects of Algorithm Design in BCS-042 course.

This practical course has been designed to provide you with hands-on experience in understanding and developing basic algorithms using the C programming language. You have already studied the theoretical concepts of algorithms, their design techniques, and complexity analysis in your theory course BCS-042. In this lab, emphasis is given on translating those concepts into practical implementations. Algorithms form the backbone of problem solving in computer science, enabling efficient and systematic solutions to real-world problems. Through this section, you will learn how to design step-by-step solutions, represent them using pseudo-code, and implement them in C. A session-wise list of practical problems is provided to help you practice various algorithmic techniques such as searching, sorting, and basic computational methods. You are advised to carefully follow the instructions and maintain proper records of your lab work. This section consists of 20 sessions, each focusing on developing logical thinking and programming skills through practical exercises.

This block consists of 1 section and is organized as follows:

Section-1comprises of 20 sessions where in session wise practical problems are given. Attempt all the problems given session wiseat Section 1.3for 20 sessions.

Happy programming!! We wish you an eventful and interesting journey to the fifth semester computer lab.

SECTION 1 INTRODUCTION TO ALGORITHM DESIGN

Structure Page Nos.

1.0 INTRODUCTION

This is the lab course, wherein you will have the hands-on experience in Algorithm Design using C Programming. You have studied the basics of an algorithm, asymptotic bounds, analysis of simple algorithms and design techniques such as Greedy Techniques, Divide and Conquer Approach and Graph Algorithms in the theory course BCS-042 Introduction to Algorithm Design.

A separate a list of 20 lab-sessions to be performed sessionwise are given in this section. Please go through the general guidelines and the program documentation guidelines carefully.

1.1 OBJECTIVES

After going through this practical course, you will be able to:

- Understand the fundamental concepts of algorithms and their role in problem solving.
- Develop logical thinking and problem-solving skills through hands-on algorithm design and implementation.
- Implement various algorithms using C programming language.
- Analyze the time and space complexity of algorithms using basic asymptotic concepts.
- Design, implement and compare various searching techniques and sorting algorithms.
- Apply divide and conquer techniques to solve problems using algorithms.
- Understand and implement greedy approaches for simple optimization problems.
- Represent data structures like graphs and perform basic traversals such as BFS and DFS.
- Debug, test and optimize C programs for correctness and efficiency.

1.2 GENERAL GUIDELINES

Following are some of the general guidelines:

- Observation book and Lab record are compulsory.
- You should attempt all problems/assignments given in the list at Section 1.3session wise.
- Use C programming language for programming.
- For the tasks describe the procedure and also present screenshots wherever applicable.
- You may seek assistance in doing the lab exercises from the concerned lab instructor. Since the assignments have credits, the lab instructor is obviously not expected to tell you how to solve these, but you may ask questions concerning the concepts.
- Add comments wherever necessary.
- The program should be interactive, general and properly documented with real Input/ Output data.
- If two or more submissions from different students appear to be of the same origin (i.e. are variants of essentially the same program), none of them will be counted. You are strongly advised not to copy somebody else's work.
- It is your responsibility to create a separate directory to store all the programs, so that nobody else can read or copy.
- The list of the programs(list of programs given at the end, session-wise) is available to you in this lab manual. For each session, you must come prepare with the algorithms and the programs written in the Observation Book. You should utilize the lab hours for executing the programs, testing for various desired outputs and enhancements of the programs.
- As soon as you have finished a lab exercise, contact one of the lab instructor / incharge in order to get the exercise evaluated and also get the signature from him/her on the Observation book.
- Completed lab assignments should be submitted in the form of a Lab Record in which you have to write the algorithm, program code along with comments and output for various inputs given.
- The total no. of lab sessions (3 hours each) are20 and the list of assignments is provided session-wise. It is important to observe the deadline given for each assignment.

Get started with the examples given in the theory course for practice and try to solve all the practical questions given in the practical sessions –session wise given below.

1.3 PRACTICAL SESSIONS

Sessionwise practical problems for 20 sessions are given below:

BLOCK – 1 UNIT-1 BASICS OF AN ALGORITHM

Session – 1: Basic Algorithm Implementation

Problem 1: Maximum of Three Numbers
Write a C program to find the maximum of three numbers using a function.

Problem 2: Sum of First N Natural Numbers
Write a program using iterative approach and compare with formula-based solution.

Problem 3: Factorial of a Number
Implement factorial using:

- Iteration
- Recursion
  Compare both approaches.

Session – 2: Pseudo-code to Program Conversion

Problem 4: Linear Search
Convert given pseudo-code into a C program to search an element in an array.

Problem 5: Count Number of Comparisons
Modify linear search to count comparisons (best and worst cases).

Problem 6: Fibonacci Series
Write a program to generate Fibonacci numbers using recursion.

Session – 3: Algorithm Complexity Understanding

Problem 7: Count Loop Operations

Write a program that counts number of operations in:

- Single loop
- Nested loop

Problem 8: Time Complexity Demonstration
Write programs showing:

- $O(n)$
- $O(n^2)$

Problem 9: Recursive Function Depth
Write a recursive function and print recursion depth.

BLOCK – 1 UNIT-2 ASYMPTOTIC BOUNDS

Session – 4: Best, Worst, Average Case

Problem 10: Linear Search Case Analysis
Implement linear search and demonstrate the Best Case and the Worst Case.

Problem 11: Comparison Counter
Modify bubble sorting and linear search programs to count the no. of comparisons.

Problem 12: Input Size vs Time
Measure execution time for increasing input sizes.

Session – 5: Asymptotic Growth Demonstration

Problem 13: Compare O(n) and O(n^2)
Write selection sort program and compare its execution time with linear search.

Problem 14: Exponential Growth Demo
Write recursive program to solve tower of Hanoi problem and analyse time.

Session – 6: Efficiency Comparison of Algorithms

Problem 15: Linear Search vs Binary Search
Implement both the programs and compare their efficiency for best, worst and average cases.

Problem 16: Operation Count Analysis
Count the operations in nested loops and derive complexity.

BLOCK – 1 UNIT-3 ANALYSIS OF SIMPLE ALGORITHMS

Session – 7: Euclid Algorithm & Number Operations

Problem 17: GCD using Euclid Algorithm

Write a C program to implement iterative version to find GCD.
(As described in Unit-3, GCD reduces problem using remainder operation)

Problem 18: Recursive GCD
Write a C program to implement recursive version of GCD and compare both.

Session – 8: Polynomial Evaluation (Horner's Rule)

Problem 19: Basic Polynomial Evaluation
Write a C program to evaluate a polynomial using direct method.

Problem 20: Horner's Rule Implementation
Write a C program to implement efficient polynomial evaluation using Horner's method
(as discussed in Unit-3).

Problem 21: Efficiency Comparison
Compare efficiency of both the methods.

Session – 9: Matrix Operations

Problem 22: Matrix Multiplication
Write a C program to multiply two $n \times n$ matrices using nested loops.

Problem 23: Count Operations
Count number of multiplications and additions used in Problem 22.

Session – 10: Searching and Sorting Algorithms

Problem 24: Matrix Transpose & Symmetry Check
Write a C program to check whether a matrix is symmetric or not.

Problem 25: Insertion Sort
Write a C program to implement insertion sort and compare its performance with selection sort.

BLOCK – 2 UNIT-1 GREEDY TECHNIQUES

SESSION – 11 (Greedy Basics)

Problem 26:Minimum Number of Currency Notes (Greedy)
Write a C program to find the minimum number of notes required to make a given amount using denominations {1, 2, 5, 10, 20, 50, 100}.

Problem 27: Activity Selection Problem
Write a C program to implement a greedy algorithm to select the maximum number of non-overlapping activities.

Problem 28:Fractional Knapsack Problem
Write a C program to maximise profit using the fractional knapsack approach.

SESSION – 12 (Advanced Greedy)

Problem 29: Job Sequencing with Deadlines
Write a C program to schedule jobs to maximise profit given deadlines.

Problem 30: Minimum Spanning Tree (Kruskal's Algorithm)
Write a C program to implement Kruskal's algorithm using greedy approach.

Problem 31: Dijkstra's Shortest Path Algorithm
Write a C program to find shortest path from a source node.

SESSION – 13 (Greedy Applications)

Problem 32: Coin Change (Greedy vs Non-Greedy Case)
Write a C program to implement greedy coin change and test cases where it fails.

Problem 33: Huffman Coding
Write a C program to Construct Huffman Tree and generate codes.

Problem 34: Prim's Algorithm
Write a C program to find MST using Prim's algorithm.

BLOCK – 2 UNIT-2 DIVIDE AND CONQUER APPROACH

SESSION – 14 (Divide & Conquer Basics)

Problem 35: Find Maximum and Minimum using Divide & Conquer
Write a C program to find *max* and *min* using recursion.

Problem 36: Power Calculation using Recursion
Write a C program to compute (a^n) using divide-and-conquer.

SESSION – 15 (Unit-2: Sorting Algorithms)

Problem 37: Merge Sort
Write a C program to implement merge sort using divide and conquer.

Problem 38: Quick Sort
Write a C program for quick sort and count comparisons for both Merge sort and Quick sort.

SESSION – 16 (Advanced Divide & Conquer)

Problem 39: Matrix Multiplication
Write a C program to implement matrix multiplication using divide and conquer.

Problem 40: Karatsuba Integer Multiplication
Write a recursive program using C language for large integer multiplication.

SESSION – 17 (Graph Representation)

Problem 41: Adjacency Matrix Representation

Write a C program to create a graph using adjacency matrix and display it. (Adjacency matrix concept from Unit-3)

Problem 42: Adjacency List Representation
Write a C program to implement graph using adjacency list.

BLOCK – 2 UNIT-3 GRAPH ALGORITHMS

SESSION – 18 (Graph Traversal – BFS)

Problem 43: Breadth First Search (BFS)
Write a C program to implement BFS using queue.

Problem 44: Shortest Path using BFS
Write a C program to find shortest path in unweighted graph.

SESSION – 19 (Graph Traversal – DFS)

Problem 45: Depth First Search (DFS)
Write a C program to implement DFS using recursion.

Problem 46: Cycle Detection
Write a C program to detect cycle in an undirected graph.

SESSION – 20(Advanced Graph Applications)

Problem 47: Path Finding
Write a C program to find all paths between two vertices.

Problem 48: Strong Connectivity Check
Write a C program to check if directed graph is strongly connected.

1.4 SUMMARY

In this section of 20 practical sessions, you learned how to design and implement basic algorithms using the C programming language. You practiced important algorithms such as searching (linear and binary search), sorting (bubble, selection, and insertion sort), and basic mathematical algorithms like GCD. You were also introduced to efficient methods such as divide and conquer (merge sort and quick sort), as well as basic concepts of greedy methods and graph traversal. This unit focused on improving logical thinking, coding skills, and understanding how to make programs efficient through testing and debugging.

1.5 FURTHER READINGS

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, PHI.

2. Foundations of Algorithms, R. Neapolitan & K. Naimipour: (D.C.Health& Company, 1996.
3. Algoritmics: The Spirit of Computing, D. Harel, Addison-WesleyPublishing Company, 1987.
4. Fundamentals of Algorithmics, G. Brassard & P. Brately, Prentice-HallInternational, 1996.
5. Fundamental Algorithms (Second Edition), D.E. Knuth, NarosaPublishing House.
6. Fundamentals of Computer Algorithms, E. Horowitz & S. Sahni, Galgotia Publications.
7. The Design and Analysis of Algorithms, Anany Levitin, Pearso,,Education, 2003.
8. Programming Languages (Second Edition) — Concepts and Constructs,Ravi Sethi, Pearson Education, Asia, 1996.